FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Keisha Belle Calibjo
Section:	AN43
Professor:	Crisola G. Tan
Date:	10/10/26

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:

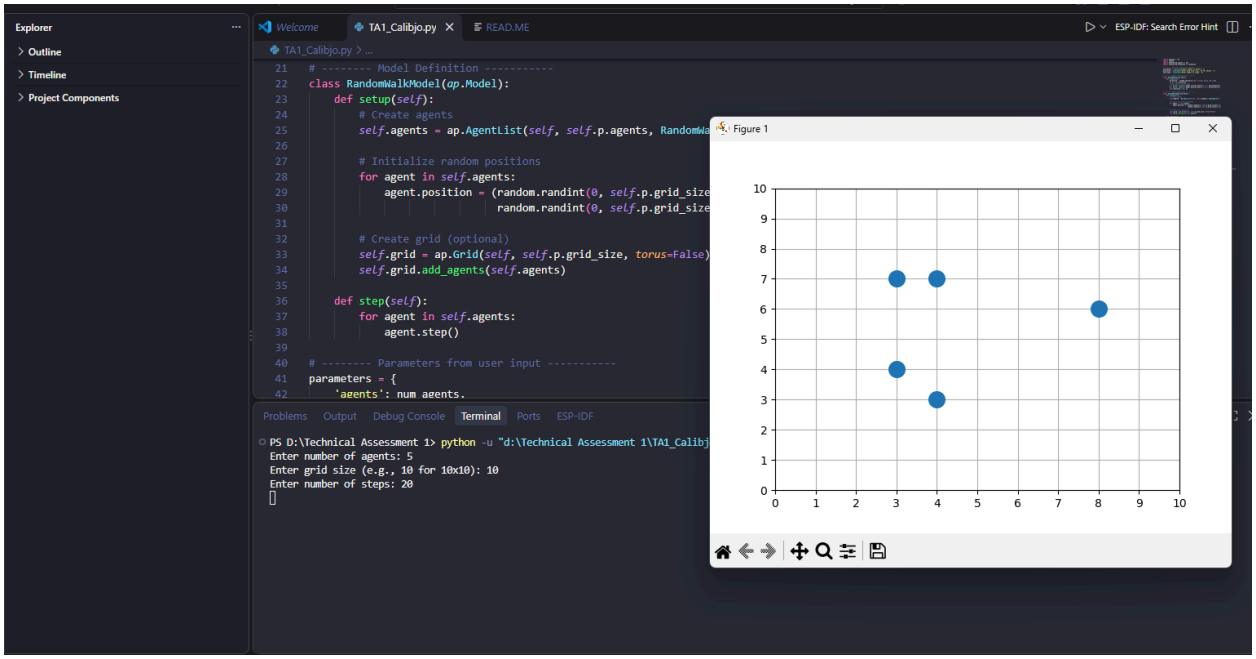
- **More agents** (e.g., 10 or 20)
- **Smaller/larger grid sizes**
- **Longer simulations** (e.g., 50 steps)

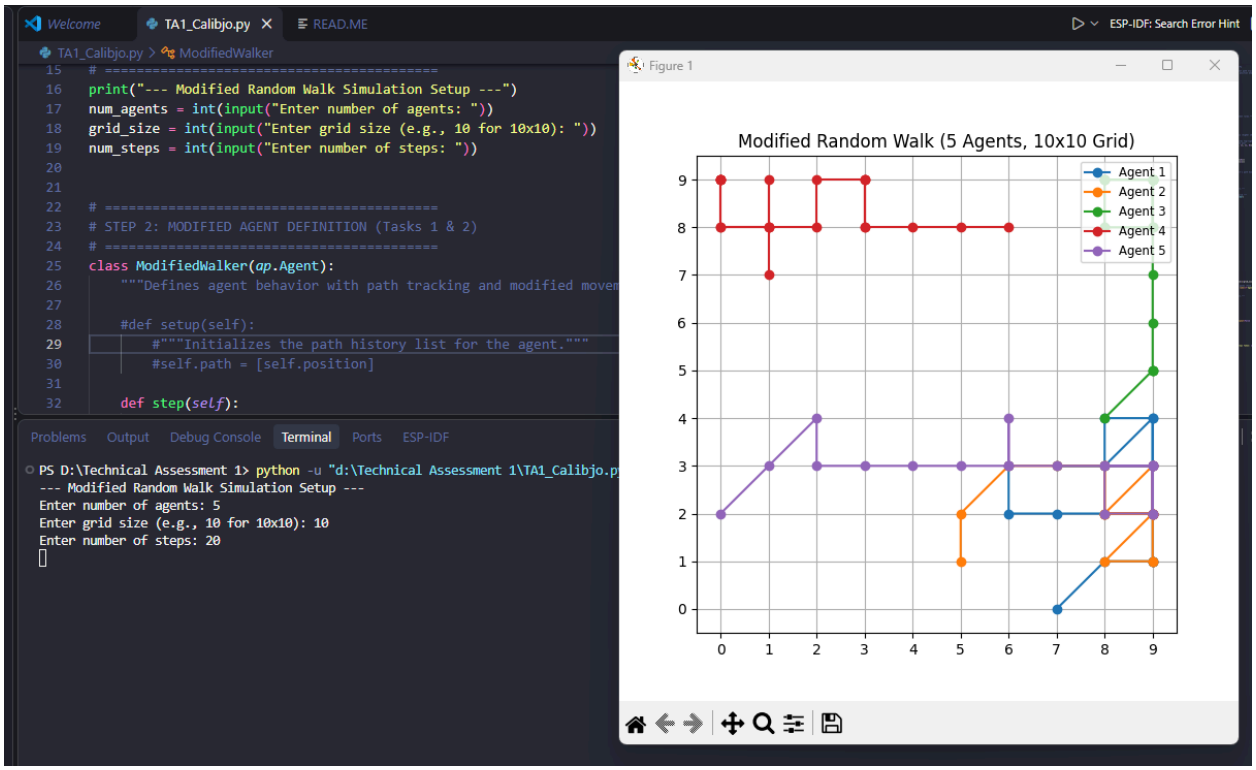
- Observe and record how the **agent movement patterns change**.

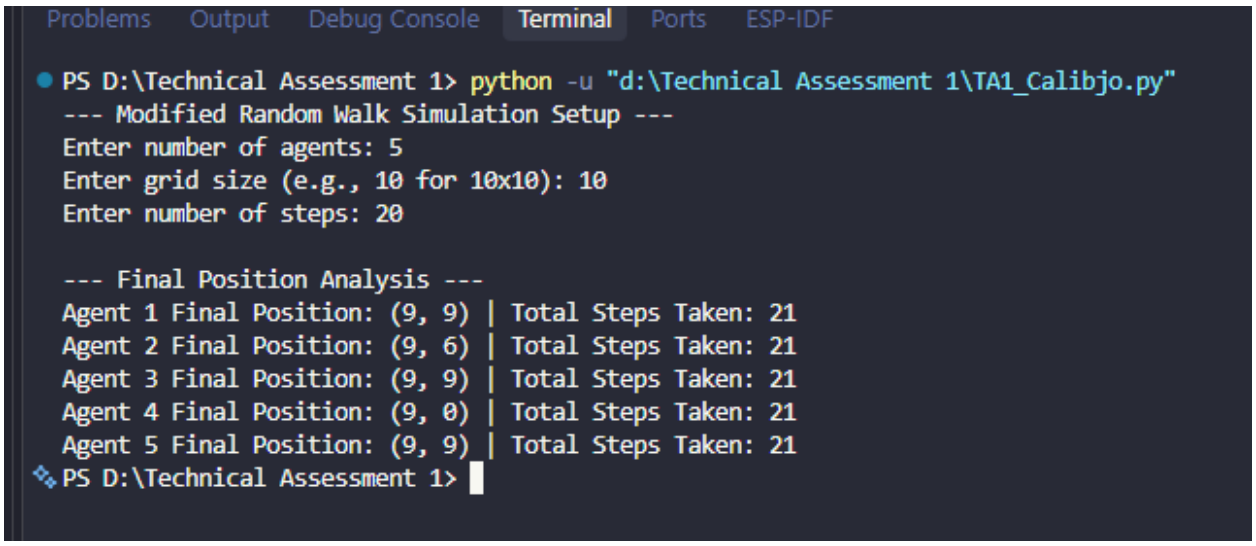
Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩








```
"""
Technical Assessment 1: Modified Random Walk Simulation using AgentPy
Course: CS0065 - Intelligent Systems
Description: Implements laboratory task modifications including agent path tracking,
```

```
                    modified movement rules (directional bias), and final
position analysis.
"""

import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ========================================
# STEP 1: GET USER INPUT FROM CONSOLE
# ========================================
print("--- Modified Random Walk Simulation Setup ---")
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))



# STEP 2: MODIFIED AGENT DEFINITION (Tasks 1 & 2)
class ModifiedWalker(ap.Agent):
     """Defines agent behavior with path tracking and modified movement
rules."""

    def step(self):
        """
        Executes a modified step:
            Task 2 Modification: Introduces a preferred directional bias
(higher chance
        to move Up and Right) alongside standard random movement.
        """
          # Weighted choices to create a preferred movement bias towards
top-right
        # (1,0)=Right, (0,1)=Up appear more frequently in the choice pool
          directions = [(1, 0), (-1, 0), (0, 1), (0, -1), (1, 1), (0, 1),
(1, 0)]
        direction = random.choice(directions)
```

```
        # Get current coordinates
        x, y = self.position

        # Keep agents inside grid boundaries (clamping)
        x = max(0, min(self.model.p.grid_size[0] - 1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1] - 1, y + direction[1]))

        # Update current position
        self.position = (x, y)

          # Task 1 Requirement: Record current position into the agent's
path history
        self.path.append(self.position)


# STEP 3: MODEL DEFINITION
class ModifiedRandomWalkModel(ap.Model):
    """Manages the simulation environment, grid, and modified agents."""

    def setup(self):
        """Initializes agents, random starting points, paths, and grid."""
        self.agents = ap.AgentList(self, self.p.agents, ModifiedWalker)

        for agent in self.agents:
            agent.position = (
                random.randint(0, self.p.grid_size[0] - 1),
                random.randint(0, self.p.grid_size[1] - 1)
            )
            # Initialize path history with starting position
            agent.path = [agent.position]

        # Create non-torus (bounded) grid
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
```

```
        """Advances the simulation model by one step."""
        self.agents.step()



# STEP 4: PARAMETERS & MODEL INITIALIZATION
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

model = ModifiedRandomWalkModel(parameters)
model.setup()


# ==========================================
# STEP 5: VISUALIZATION WITH TRAILING PATHS
# ==========================================
fig, ax = plt.subplots(figsize=(6, 6))
ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
ax.set_xticks(range(model.p.grid_size[0]))
ax.set_yticks(range(model.p.grid_size[1]))
ax.grid(True)
ax.set_title(f"Modified     Random     Walk     ({num_agents}     Agents,
{grid_size}x{grid_size} Grid)")

# Create plot line handles for each agent to display their tracked paths
(Task 1)
lines   =   [ax.plot([],   [],   marker='o',   linestyle='-',   linewidth=1.5,
label=f'Agent {i+1}')[0]
         for i in range(model.p.agents)]

def update(frame):
    """Animation update function showing real-time path trajectories."""
    model.step()
```

```
    # Update each agent's path line data
    for i, agent in enumerate(model.agents):
        x_coords = [p[0] for p in agent.path]
        y_coords = [p[1] for p in agent.path]
        lines[i].set_data(x_coords, y_coords)

    return lines

ani = animation.FuncAnimation(
         fig,  update,  frames=model.p.steps,  interval=300,  blit=False,
repeat=False
)

plt.legend(loc='upper right', fontsize='small')
plt.show()



# STEP 6: FINAL POSITION ANALYSIS (Task 3)
print("\n--- Final Position Analysis ---")
for i, agent in enumerate(model.agents):
     print(f"Agent {i+1} Final Position: {agent.position} | Total Steps
Taken: {len(agent.path) - 1}")
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. When the agent density is extremely high relative to the grid dimensions, spatial congestion occurs. Agents frequently crowd into the same regions or boundary cells, limiting free movement and increasing overlap, which simulates high-density crowd dynamics or molecular crowding.

2. As the number of simulation steps increases, agents have more time to diffuse outward from their initial starting points. This increases the overall spatial dispersion, expanding the coverage area and variance of agent positions across the grid.
3. No, agents never move outside the grid. This is prevented by the boundary-clamping logic in the RandomWalker.step() method (max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))), which constrains coordinates within valid grid index limits.
4. Random walk models serve as foundational frameworks for simulating molecular diffusion in chemistry/physics, foraging patterns of animals in biology, price fluctuations in financial markets, and pedestrian evacuation or crowd flow in architectural spaces.